NOUS AI ACADEMY / INTENSIVE FILE

TECHNICAL LEARNING FILE

DEEP-15

Deep Learning with PyTorch

Transparent neural training loops

FORMAT	LENGTH	ACCESS
INTENSIVE FILE	50 PAGES	OFFLINE

Edition 2 - original curriculum - editorial review recommended

FILE / ORIENTATION

Purpose

01 FILE GUIDANCE

This optional advanced guide does not gate the core learning path. Apply deep learning with pytorch with explicit assumptions, tests, tradeoffs, and limitations.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

FILE / ORIENTATION

Prerequisites

01 FILE GUIDANCE

Complete the related core track or pass its diagnostic.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

FILE / ORIENTATION

Study Method

01 FILE GUIDANCE

For each unit, explain the concept, follow the workflow, reproduce the example, analyze a failure, and complete the applied lab. Record evidence locally.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

UNIT 01 / CONCEPT

Tensors: Concept

01 OBJECTIVE

Explain and apply tensors using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Tensors is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Deep Learning with PyTorch, the terms Tensors are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should tensors be used, and what observable evidence would make that choice defensible? Build a small local example that applies tensors, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Tensors in one precise sentence and name one assumption.
2. Create a two-step example of tensors and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / WORKFLOW

Tensors: Workflow

01 OBJECTIVE

Explain and apply tensors using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Tensors is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to tensors.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies tensors, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Tensors in one precise sentence and name one assumption.
2. Create a two-step example of tensors and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / WORKED EXAMPLE

Tensors: Worked Example

01 OBJECTIVE

Explain and apply tensors using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies tensors, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns tensors into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Tensors in one precise sentence and name one assumption.
2. Create a two-step example of tensors and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / FAILURE ANALYSIS

Tensors: Failure Analysis

01 OBJECTIVE

Explain and apply tensors using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how tensors can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to tensors. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Tensors in one precise sentence and name one assumption.
2. Create a two-step example of tensors and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / APPLIED LAB

Tensors: Applied Lab

01 OBJECTIVE

Explain and apply tensors using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of tensors into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies tensors, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Tensors in one precise sentence and name one assumption.
2. Create a two-step example of tensors and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / CONCEPT

Autograd: Concept

01 OBJECTIVE

Explain and apply autograd using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Autograd is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Deep Learning with PyTorch, the terms Autograd are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should autograd be used, and what observable evidence would make that choice defensible? Build a small local example that applies autograd, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Autograd in one precise sentence and name one assumption.
2. Create a two-step example of autograd and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / WORKFLOW

Autograd: Workflow

01 OBJECTIVE

Explain and apply autograd using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Autograd is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to autograd.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies autograd, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Autograd in one precise sentence and name one assumption.
2. Create a two-step example of autograd and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / WORKED EXAMPLE

Autograd: Worked Example

01 OBJECTIVE

Explain and apply autograd using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies autograd, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns autograd into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Autograd in one precise sentence and name one assumption.
2. Create a two-step example of autograd and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / FAILURE ANALYSIS

Autograd: Failure Analysis

01 OBJECTIVE

Explain and apply autograd using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how autograd can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to autograd. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Autograd in one precise sentence and name one assumption.
2. Create a two-step example of autograd and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / APPLIED LAB

Autograd: Applied Lab

01 OBJECTIVE

Explain and apply autograd using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of autograd into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies autograd, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Autograd in one precise sentence and name one assumption.
2. Create a two-step example of autograd and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / CONCEPT

Modules: Concept

01 OBJECTIVE

Explain and apply modules using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Modules is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Deep Learning with PyTorch, the terms Modules are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should modules be used, and what observable evidence would make that choice defensible? Build a small local example that applies modules, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Modules in one precise sentence and name one assumption.
2. Create a two-step example of modules and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / WORKFLOW

Modules: Workflow

01 OBJECTIVE

Explain and apply modules using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Modules is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to modules.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies modules, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Modules in one precise sentence and name one assumption.
2. Create a two-step example of modules and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / WORKED EXAMPLE

Modules: Worked Example

01 OBJECTIVE

Explain and apply modules using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies modules, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns modules into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Modules in one precise sentence and name one assumption.
2. Create a two-step example of modules and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / FAILURE ANALYSIS

Modules: Failure Analysis

01 OBJECTIVE

Explain and apply modules using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how modules can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to modules. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Modules in one precise sentence and name one assumption.
2. Create a two-step example of modules and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / APPLIED LAB

Modules: Applied Lab

01 OBJECTIVE

Explain and apply modules using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of modules into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies modules, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Modules in one precise sentence and name one assumption.
2. Create a two-step example of modules and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / CONCEPT

Datasets and Loaders: Concept

01 OBJECTIVE

Explain and apply datasets and loaders using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Datasets and Loaders is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Deep Learning with PyTorch, the terms Datasets, Loaders are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should datasets and loaders be used, and what observable evidence would make that choice defensible? Build a small local example that applies datasets and loaders, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Datasets and Loaders in one precise sentence and name one assumption.
2. Create a two-step example of datasets and loaders and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / WORKFLOW

Datasets and Loaders: Workflow

01 OBJECTIVE

Explain and apply datasets and loaders using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Datasets and Loaders is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to datasets and loaders.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies datasets and loaders, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Datasets and Loaders in one precise sentence and name one assumption.
2. Create a two-step example of datasets and loaders and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / WORKED EXAMPLE

Datasets and Loaders: Worked Example

01 OBJECTIVE

Explain and apply datasets and loaders using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies datasets and loaders, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns datasets and loaders into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Datasets and Loaders in one precise sentence and name one assumption.
2. Create a two-step example of datasets and loaders and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / FAILURE ANALYSIS

Datasets and Loaders: Failure Analysis

01 OBJECTIVE

Explain and apply datasets and loaders using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how datasets and loaders can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to datasets and loaders. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Datasets and Loaders in one precise sentence and name one assumption.
2. Create a two-step example of datasets and loaders and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / APPLIED LAB

Datasets and Loaders: Applied Lab

01 OBJECTIVE

Explain and apply datasets and loaders using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of datasets and loaders into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies datasets and loaders, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Datasets and Loaders in one precise sentence and name one assumption.
2. Create a two-step example of datasets and loaders and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / CONCEPT

Training Loops: Concept

01 OBJECTIVE

Explain and apply training loops using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Training Loops is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Deep Learning with PyTorch, the terms Training, Loops are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should training loops be used, and what observable evidence would make that choice defensible? Build a small local example that applies training loops, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Training Loops in one precise sentence and name one assumption.
2. Create a two-step example of training loops and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / WORKFLOW

Training Loops: Workflow

01 OBJECTIVE

Explain and apply training loops using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Training Loops is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to training loops.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies training loops, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Training Loops in one precise sentence and name one assumption.
2. Create a two-step example of training loops and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / WORKED EXAMPLE

Training Loops: Worked Example

01 OBJECTIVE

Explain and apply training loops using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies training loops, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns training loops into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Training Loops in one precise sentence and name one assumption.
2. Create a two-step example of training loops and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / FAILURE ANALYSIS

Training Loops: Failure Analysis

01 OBJECTIVE

Explain and apply training loops using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how training loops can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to training loops. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Training Loops in one precise sentence and name one assumption.
2. Create a two-step example of training loops and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / APPLIED LAB

Training Loops: Applied Lab

01 OBJECTIVE

Explain and apply training loops using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of training loops into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies training loops, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Training Loops in one precise sentence and name one assumption.
2. Create a two-step example of training loops and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / CONCEPT

Validation: Concept

01 OBJECTIVE

Explain and apply validation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Validation is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Deep Learning with PyTorch, the terms Validation are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should validation be used, and what observable evidence would make that choice defensible? Build a small local example that applies validation, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Validation in one precise sentence and name one assumption.
2. Create a two-step example of validation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / WORKFLOW

Validation: Workflow

01 OBJECTIVE

Explain and apply validation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Validation is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to validation.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies validation, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Validation in one precise sentence and name one assumption.
2. Create a two-step example of validation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / WORKED EXAMPLE

Validation: Worked Example

01 OBJECTIVE

Explain and apply validation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies validation, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns validation into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Validation in one precise sentence and name one assumption.
2. Create a two-step example of validation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / FAILURE ANALYSIS

Validation: Failure Analysis

01 OBJECTIVE

Explain and apply validation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how validation can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to validation. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Validation in one precise sentence and name one assumption.
2. Create a two-step example of validation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / APPLIED LAB

Validation: Applied Lab

01 OBJECTIVE

Explain and apply validation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of validation into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies validation, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Validation in one precise sentence and name one assumption.
2. Create a two-step example of validation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / CONCEPT

Checkpoints: Concept

01 OBJECTIVE

Explain and apply checkpoints using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Checkpoints is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Deep Learning with PyTorch, the terms Checkpoints are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should checkpoints be used, and what observable evidence would make that choice defensible? Build a small local example that applies checkpoints, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Checkpoints in one precise sentence and name one assumption.
2. Create a two-step example of checkpoints and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / WORKFLOW

Checkpoints: Workflow

01 OBJECTIVE

Explain and apply checkpoints using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Checkpoints is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to checkpoints.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies checkpoints, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Checkpoints in one precise sentence and name one assumption.
2. Create a two-step example of checkpoints and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / WORKED EXAMPLE

Checkpoints: Worked Example

01 OBJECTIVE

Explain and apply checkpoints using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies checkpoints, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns checkpoints into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Checkpoints in one precise sentence and name one assumption.
2. Create a two-step example of checkpoints and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / FAILURE ANALYSIS

Checkpoints: Failure Analysis

01 OBJECTIVE

Explain and apply checkpoints using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how checkpoints can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to checkpoints. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Checkpoints in one precise sentence and name one assumption.
2. Create a two-step example of checkpoints and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / APPLIED LAB

Checkpoints: Applied Lab

01 OBJECTIVE

Explain and apply checkpoints using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of checkpoints into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies checkpoints, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Checkpoints in one precise sentence and name one assumption.
2. Create a two-step example of checkpoints and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / CONCEPT

Performance: Concept

01 OBJECTIVE

Explain and apply performance using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Performance is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Deep Learning with PyTorch, the terms Performance are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should performance be used, and what observable evidence would make that choice defensible? Build a small local example that applies performance, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Performance in one precise sentence and name one assumption.
2. Create a two-step example of performance and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / WORKFLOW

Performance: Workflow

01 OBJECTIVE

Explain and apply performance using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Performance is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to performance.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies performance, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Performance in one precise sentence and name one assumption.
2. Create a two-step example of performance and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / WORKED EXAMPLE

Performance: Worked Example

01 OBJECTIVE

Explain and apply performance using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies performance, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns performance into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Performance in one precise sentence and name one assumption.
2. Create a two-step example of performance and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / FAILURE ANALYSIS

Performance: Failure Analysis

01 OBJECTIVE

Explain and apply performance using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how performance can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to performance. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Performance in one precise sentence and name one assumption.
2. Create a two-step example of performance and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / APPLIED LAB

Performance: Applied Lab

01 OBJECTIVE

Explain and apply performance using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of performance into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies performance, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Performance in one precise sentence and name one assumption.
2. Create a two-step example of performance and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / CONCEPT

Debugging: Concept

01 OBJECTIVE

Explain and apply debugging using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Debugging is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Deep Learning with PyTorch, the terms Debugging are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should debugging be used, and what observable evidence would make that choice defensible? Build a small local example that applies debugging, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Debugging in one precise sentence and name one assumption.
2. Create a two-step example of debugging and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / WORKFLOW

Debugging: Workflow

01 OBJECTIVE

Explain and apply debugging using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Debugging is a central part of deep learning with pytorch because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to debugging.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies debugging, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Debugging in one precise sentence and name one assumption.
2. Create a two-step example of debugging and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / WORKED EXAMPLE

Debugging: Worked Example

01 OBJECTIVE

Explain and apply debugging using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies debugging, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns debugging into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Debugging in one precise sentence and name one assumption.
2. Create a two-step example of debugging and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / FAILURE ANALYSIS

Debugging: Failure Analysis

01 OBJECTIVE

Explain and apply debugging using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how debugging can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to debugging. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Debugging in one precise sentence and name one assumption.
2. Create a two-step example of debugging and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / APPLIED LAB

Debugging: Applied Lab

01 OBJECTIVE

Explain and apply debugging using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of debugging into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies debugging, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Debugging in one precise sentence and name one assumption.
2. Create a two-step example of debugging and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

FILE / ORIENTATION

Deep-Dive Capstone

01 FILE GUIDANCE

Build a compact, reproducible artifact demonstrating deep learning with pytorch. Include assumptions, a baseline, tests, failure analysis, results, limitations, and instructions for repeating the work offline.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.